Universidad Diego Portales
ESCUELA DE INFORMÁTICA &
TELECOMUNICACIONES

ESTRUCTURAS DE DATOS &
ANÁLISIS DE ALGORITMOS

Laboratorio 1: Análisis empírico y teórico de algoritmos

Autores:
Renato Parra
Matias Herrera

Sección 5

Profesor:
Marcos Fantoval

Fecha de entrega
22-03-2026

Índice

1. Introducción **2**

2. Etapa 1: Análisis empírico **2**
2.1. (I) Estructura del software 2
2.2. (II) Desafíos enfrentados 2
2.3. (III) Análisis empírico 3

3. Etapa 2: Resultados en Excel **4**

4. Etapa 3: Análisis teórico **5**
4.1. Método 1: Dos Punteros (`cuentaPares`) 6
4.2. Método 2: Búsqueda Binaria (`cuentaPares2`) 6
4.3. Método 3: Fuerza Bruta (`cuentaPares3`) 7

5. Etapa 4: Comparación teoría vs. experimento **7**

6. Conclusiones **8**

1. Introducción

El presente informe detalla el análisis empírico y la demostración teórica de complejidad asintótica (Big-O) de tres algoritmos diseñados para resolver un problema computacional específico: determinar la cantidad de pares de elementos dentro de una colección cuya diferencia absoluta es exactamente igual a una constante escalar k.

El objetivo principal radica en evaluar el orden de crecimiento temporal de cada enfoque algorítmico frente a entradas de tamaño escalable, desarrollando perfiles de ejecución sobre la máquina virtual de Java (JVM). Estos tiempos de procesamiento fueron tabulados y analizados mediante herramientas de cálculo para estimar empíricamente sus límites asintóticos a través del método de la tasa de duplicación y visualizaciones de dispersión log-log, corroborando posteriormente la evidencia práctica con el análisis matemático formal del código fuente.

2. Etapa 1: Análisis empírico

2.1. (I) Estructura del software

La arquitectura del proyecto se divide estructuralmente en dos componentes principales orientados a objetos. En primer lugar, la clase `Result.java` encapsula las tres lógicas algorítmicas sometidas a evaluación (`cuentaPares`, `cuentaPares2` y `cuentaPares3`).

En segundo lugar, el archivo `main.java` actúa como controlador del flujo de experimentación, garantizando el cumplimiento de los estándares de la Universidad de Princeton. Para la generación procedimental de los datos de prueba, se utilizó la clase `StdRandom` fijando una semilla (seed) para asegurar que el experimento sea determinista y reproducible, mientras que la unicidad de los elementos se garantizó mediante la estructura `HashSet`.

El diseño del ciclo de experimentación se basó fielmente en el modelo arquitectónico de la clase `DoublingRatio`, escalando el tamaño de la instancia en potencias de dos. La recolección de métricas de tiempo aisladas se efectuó utilizando la clase `StopwatchCPU`, y los resultados tabulares fueron agrupados en archivos CSV categorizados por volumen de entrada (n) utilizando los flujos de salida de la clase `Out`. Para mantener la integridad experimental, el orquestador clona la lista de números antes de invocar cada método.

2.2. (II) Desafíos enfrentados

Durante el desarrollo del laboratorio, las barreras iniciales estuvieron centradas principalmente en la gestión del entorno de trabajo. Hubo una curva de aprendizaje considerable en la adopción del sistema de control de versiones distribuido (Git y

GitHub), enfrentando dificultades operativas relacionadas con la sincronización de ramas y la prevención de sobrescritura de historiales entre colaboradores.

A nivel de software, el principal impedimento radicó en la resolución de dependencias del entorno de desarrollo (IDE). En diversas ocasiones, el compilador arrojó excepciones de tipo `ClassNotFoundException` debido a la incorrecta vinculación de los paquetes externos (el archivo `algs4.jar` requerido para `StdRandom` y `StopwatchCPU`). Estos conflictos fueron solucionados a través de la realización del proyecto.

2.3. (III) Análisis empírico

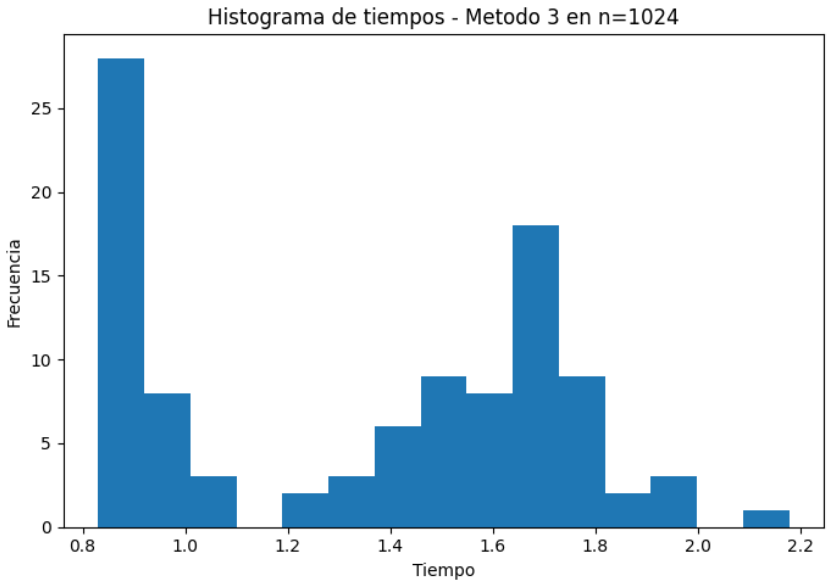

Figura 1: Histograma de distribución de los tiempos del método de fuerza bruta para $n = 1024$.

El marco experimental se definió estableciendo un dominio de tamaños de entrada $n \in \{32, 64, 128, 256, 512, 1024\}$, con 100 instancias aleatorias por tamaño para mitigar el ruido del sistema operativo.

Al analizar los resultados empíricos, los promedios e histogramas demuestran la disparidad de las eficiencias estructurales. Para el método de fuerza bruta, la distribución de tiempos se desplaza drásticamente hacia la derecha conforme n aumenta, indicando una rápida degradación del rendimiento. A través de la estrategia de duplicación, se evidencia que los Métodos 1 y 2 mantienen tiempos marginales de ejecución, mientras que el Método 3 cuadruplica su tiempo de ejecución en cada salto de tamaño.

3. Etapa 2: Resultados en Excel

n	Promedio1	Promedio2	Promedio3	Ratio1	Log2(ratio1)	Ratio2	Log2(ratio2)	Ratio3	Log2(ratio3)
32	0.0766	0.057333	0.178383						
64	0.08309	0.058978	0.155967	1.084735	0.117343	1.028696	0.040817	0.874337	-0.19374
128	0.072085	0.060946	0.029424	0.867546	-0.20499	1.033371	0.047358	0.188657	-2.40616
256	0.114662	0.119507	0.102975	1.590653	0.66962	1.960864	0.971489	3.499688	1.807226
512	0.090322	0.125521	0.361108	0.787721	-0.34424	1.050328	0.070841	3.506744	1.810132
1024	0.159811	0.205076	1.340908	1.769355	0.823224	1.633798	0.70823	3.713312	1.892706

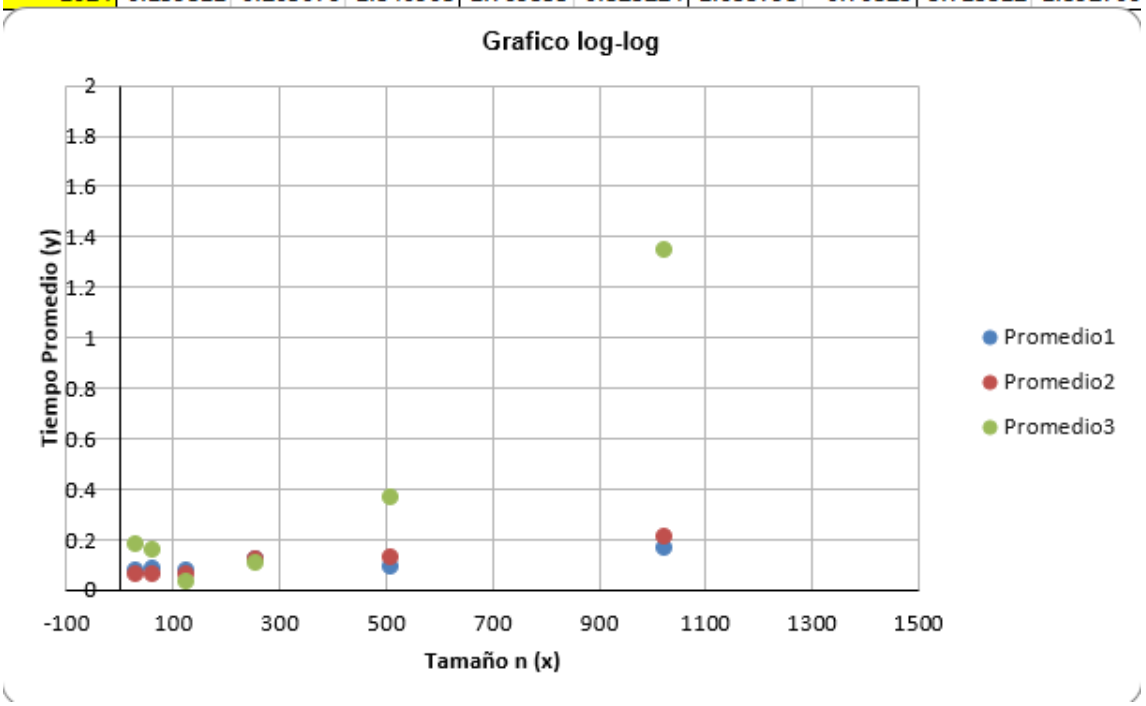

Figura 2: Resumen tabular de promedios, ratios y límite asintótico calculado a partir de los microdatos.

En el archivo Excel se procesaron computacionalmente los microdatos obtenidos. Cada hoja dedicada a un tamaño n contiene los 100 tiempos medidos y sus respectivas estadísticas descriptivas (Mínimo, Máximo, Promedio, Desviación Estándar y Coeficiente de Variación), acompañados de histogramas de distribución.

En la pestaña resumen se tabularon las métricas consolidadas, calculando explícitamente el ratio de crecimiento $T(2n)/T(n)$ y su logaritmo base 2 ($\log_2(\text{ratio})$).

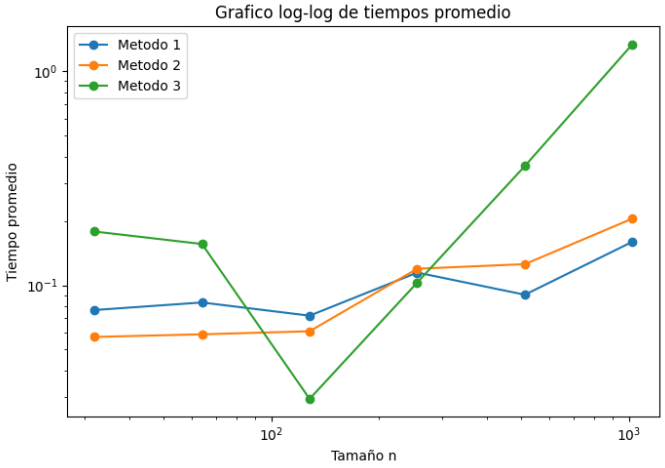

Figura 3: Gráfico log-log de los tiempos promedio, evidenciando el contraste de pendientes.

Además, se generó un gráfico de dispersión log-log. En este tipo de gráficas, el exponente polinomial del algoritmo se manifiesta directamente como la pendiente de la recta. La serie de datos correspondiente al Método 3 traza una pendiente marcadamente pronunciada. Por el contrario, las métricas de los enfoques 1 y 2 se agrupan en la zona inferior de la gráfica trazando trayectorias significativamente más planas, demostrando visualmente un orden de crecimiento notoriamente inferior al cuadrático.

4. Etapa 3: Análisis teórico

A continuación, se presenta la deducción formal de la complejidad temporal (Big-O) justificando las estructuras de control que dominan el costo asintótico en cada implementación.

4.1. Método 1: Dos Punteros (`cuentaPares`)

```java
public static int cuentaPares(int pN, List<Integer> pValores, int pK) {
  int izq = 0;
  int der = 0;
  int res = 0;

  pValores.sort(null);  O(NlogN)
  while (der < pValores.size()) {  O(N)
    if (pValores.get(izq) + pK > pValores.get(der)) {
      der += 1;
    } else if (pValores.get(izq) + pK == pValores.get(der)) {
      res += 1;
      izq += 1;
      der += 1;
    } else if (pValores.get(izq) + pK < pValores.get(der)) {
      izq += 1;
    }
  }
  return res;  O(NlogN) + O(N) = O(NlogN
}
```

Figura 4: Implementación del método de dos punteros.

El algoritmo se divide en dos etapas computacionales. Primero, la invocación de `sort()` ejecuta un ordenamiento nativo que garantiza un límite de $O(n \log n)$. Segundo, se emplea un bucle `while` con dos variables de estado (`izq` y `der`). Dado que ambas variables son monotónicamente crecientes y están acotadas por el tamaño n, el número máximo de incrementos combinados es $2n$, resultando en un recorrido de costo $O(n)$. Al sumar ambas etapas, el término de ordenamiento domina cuando $n \to \infty$. Ignorando constantes, la complejidad teórica final es $O(n \log n)$.

4.2. Método 2: Búsqueda Binaria (`cuentaPares2`)

```java
public static int cuentaPares2(int pN, List<Integer> pValores, int pK) {
  int res = 0;
  pValores.sort(null);  O(NlogN)
  for (int i = 0; i < pValores.size(); i++) {  O(N)
    if (Collections.binarySearch(pValores, pValores.get(i) + pK) >= 0) {  O(logN)
      res += 1;
    }
  }
  return res;  O(NlogN) + O(N) + O(logN) = O(NlogN)
}
```

Figura 5: Implementación basada en búsqueda binaria múltiple.

Este método utiliza una estrategia de preprocesamiento seguida de búsquedas dicotómicas. El ordenamiento inicial cuesta $O(n \log n)$. Luego, realiza un ciclo `for` de

n iteraciones. En cada iteración, ejecuta `Collections.binarySearch`, cuya complejidad es $O(\log n)$. Esto produce un costo para la fase de búsqueda de $n \times O(\log n) = O(n \log n)$. Sumando ambos procesos dominantes, la complejidad teórica final se consolida en $O(n \log n)$.

4.3. Método 3: Fuerza Bruta (`cuentaPares3`)

```
public static int cuentaPares3(int pN, List<Integer> pValores, int pK) {
  int res = 0;
  for (int i = 0; i < pValores.size(); i++) {
    for (int j = i + 1; j < pValores.size(); j++) {
      if (Math.abs(pValores.get(i) - pValores.get(j)) == pK) {
        res += 1;
      }
    }
  }
  return res;
}
```

O(N'2)

Figura 6: Implementación mediante evaluación exhaustiva.

Aborda el problema sin preprocesamiento de la estructura, recurriendo a una comparación uno a uno. Implementa dos bucles `for` anidados: el iterador exterior recorre desde 0 hasta $n-1$, y el interior desde $i+1$. El número de evaluaciones de costo constante corresponde a la sumatoria aritmética:

$$\sum_{i=0}^{n-1}(n-1-i) = \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

Descartando factores constantes y términos de menor grado cuando $n \to \infty$, el tiempo de ejecución escala de forma puramente cuadrática. Su complejidad teórica es $O(n^2)$.

5. Etapa 4: Comparación teoría vs. experimento

Para verificar si la matemática se cumple en el entorno de ejecución real, contrastamos las estimaciones Big-O con los valores experimentales extraídos del cálculo de $\log_2(\text{ratio})$ al duplicar el tamaño.

Tamaño n	**Log2(Ratio) M1**	**Log2(Ratio) M2**	**Log2(Ratio) M3**
64	0.117	0.040	-0.193
128	-0.204	0.047	-2.406
256	0.669	0.971	1.807
512	-0.344	0.070	1.810
1024	0.823	0.708	**1.892**

Cuadro 1: Contrastación de valores experimentales de $\log_2(ratio)$ por tamaño.

Como se puede apreciar, la contrastación arrojó resultados plenamente consistentes con la teoría. El modelo teórico predecía que el método de fuerza bruta presentaría un comportamiento cuadrático. En la transición computacional hasta $n = 1024$, el $\log_2$(ratio) del Método 3 converge matemáticamente a un valor de **1.892**. Esta magnitud corrobora casi a la perfección el exponente 2 estipulado por $O(n^2)$.

Por otro lado, la teoría asigna a los métodos 1 y 2 un orden de $O(n \log n)$. Si bien los valores empíricos en la tabla oscilan y presentan ruido (valores negativos o cercanos a cero en tamaños pequeños), esto se fundamenta en causas razonables del hardware. Para tamaños $n \leq 1024$, los algoritmos 1 y 2 son tan eficientes que los tiempos de ejecución reales son eclipsados por las constantes ocultas de la máquina. A pesar del ruido subyacente, la disparidad de crecimiento general frente al Método 3 confirma la enorme ventaja del preprocesamiento de datos observada en el análisis asintótico.

6. Conclusiones

El desarrollo de esta investigación demostró de manera tangible el impacto crítico que posee el diseño estructural de un algoritmo sobre su rendimiento computacional. Frente al mismo desafío lógico, enfoques algorítmicamente distintos generaron escalas de eficiencia abismalmente diferentes.

Se constató empíricamente que la técnica exhaustiva de fuerza bruta posee una escalabilidad insostenible para entradas masivas, confirmando su naturaleza cuadrática $O(n^2)$ por su carencia de optimización. Por el contrario, la eficiencia de los algoritmos presentes en los Métodos 1 y 2 permitió que se realizaran búsquedas mucho más rápidas, manteniendo el tiempo total de ejecución en $O(n \log n)$.

Finalmente, el laboratorio resalta la vital importancia de evaluar el software tanto en el papel como en el procesador. La notación Big-O es una métrica absolutamente confiable para proyectar la escalabilidad a nivel matemático. Sin embargo, como se evidenció en la medición ruidosa de los algoritmos de alta eficiencia, la experimentación empírica es esencial para conocer las limitantes de hardware, los tiempos constantes de las plataformas subyacentes (JVM) y el rendimiento efectivo real de las soluciones tecnológicas frente a cargas variables de trabajo.